

Clase Práctica : Ruteo

Tomás F. Melli

October 2025

Índice

1	Introducción	2
1.1	Cómo hace el forwarding el switch?	3
1.2	Ruteo como grafo	3
2	Ejercicio 1	4
3	Protocolos de Ruteo Interno	6
3.1	Vector de Distancia - RIP	6
3.1.1	Características generales	6
3.1.2	Paquete RIP	6
3.1.3	Funcionamiento del algoritmo	6
3.1.4	Ejercicio 2	6
3.1.5	La red ya convergió.	7
3.1.6	Propiedades	8
3.1.7	Actualizaciones de la tabla de enrutamiento	8
3.1.8	Problema de conteo al infinito	8
3.2	Ejercicio 3 : Topología de Red	9
3.3	Estado de enlace - OSPF	10
3.3.1	Características principales	10
3.3.2	Funcionamiento	11
3.3.3	Ejercicio 4 : OSPF	12
3.3.4	Ejercicio 5	13

1 Introducción

En el contexto del funcionamiento interno de los routers, suelen mencionarse dos estructuras clave: la **tabla de routing** y la **tabla de forwarding**. Aunque a menudo se las usa como sinónimos, cumplen funciones diferentes dentro del proceso de encaminamiento y reenvío de paquetes.

- **Tabla de Routing:**

- Contiene entradas del tipo $\langle \text{Red}, \text{Costo}, \text{Próximo Salto} \rangle$.
- Es el resultado de un **proceso basado en software**, producto de la ejecución de un *algoritmo de enrutamiento* (por ejemplo, Distance Vector o Link State).
- Pertenece exclusivamente al **nivel de Red**.
- Es la **precursora** de la tabla de forwarding: a partir de la información que almacena, se construye una versión optimizada para el reenvío.
- Puede contener información adicional sobre cómo fue calculada cada ruta (por ejemplo, métrica, protocolo que la originó, o interfaces asociadas).
- Es posible que incluya **entradas repetidas** o redundantes, reflejando distintas alternativas hacia un mismo destino.

- **Tabla de Forwarding:**

- Contiene entradas simplificadas del tipo $\langle \text{Red}, \text{Próximo Salto} \rangle$.
- Corresponde a un **proceso mecánico**, orientado a la velocidad y eficiencia del reenvío de paquetes.
- Se usa directamente en el **plano de datos**, es decir, durante la toma de decisiones sobre a qué interfaz o vecino se debe reenviar cada paquete entrante.
- Se aplica tanto en el **nivel de Enlace** como en el **nivel de Red**, según la arquitectura del dispositivo.
- No contiene entradas repetidas: sólo mantiene la mejor ruta hacia cada red destino, optimizando la búsqueda y reduciendo la latencia.

- **Relación entre ambas:**

- La *tabla de routing* es generada y mantenida por los protocolos de enrutamiento dinámico o por configuraciones estáticas.
- A partir de ella, el sistema operativo del router construye la *tabla de forwarding*, que representa la versión práctica y optimizada usada para el tráfico real.
- En otras palabras, el **plano de control** (routing) decide qué rutas existen, mientras que el **plano de datos** (forwarding) decide cómo enviar los paquetes en tiempo real.

Notación

Tabla de Routing

Network	Cost	Next hop
172.16.5.0/24	0	IF 0/0
10.4.2.0/27	0	IF 0/1
192.168.2.0/26	1	10.4.2.25
Default	2	10.4.2.25

Tabla de Forwarding

Network	Next hop
172.16.5.0/24	IF 0/0
10.4.2.0/27	IF 0/1
192.168.2.0/26	10.4.2.25
Default	10.4.2.25

Network (Red)	Next Hop (Próximo salto)
Red destino	- Interface de salida. (si la red esta directamente conectada) - Dirección IP del próximo salto. (Si la red destino es una red remota)

1.1 Cómo hace el forwarding el switch?

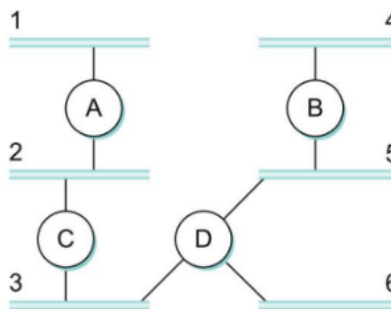
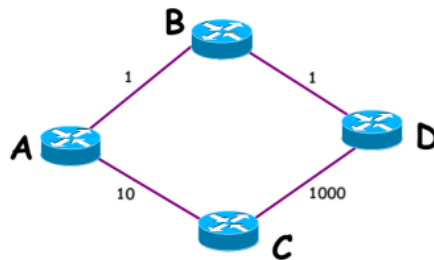
En el caso de un **switch**, el proceso de forwarding se realiza en el **nivel de Enlace de Datos (Capa 2 del modelo OSI)**. El switch examina la **dirección MAC de destino** presente en la trama Ethernet y consulta su *tabla de conmutación (MAC table)* para determinar por qué puerto debe reenviar la trama.

- Si la dirección MAC está registrada en la tabla, el switch reenvía la trama únicamente por el puerto correspondiente.
- Si la dirección no está presente, el switch difunde la trama por todos sus puertos (excepto aquel por el que la recibió), utilizando el mecanismo de *flooding*.

Este proceso es puramente mecánico y extremadamente rápido, ya que los switches modernos implementan la tabla de forwarding en hardware (por ejemplo, mediante TCAM —*Ternary Content Addressable Memory*—) para lograr una conmutación casi instantánea. A diferencia de los routers, el switch no ejecuta algoritmos de ruteo complejos: simplemente aplica las reglas de forwarding ya aprendidas por observación del tráfico.

1.2 Ruteo como grafo

El proceso de **ruteo** puede entenderse, desde una perspectiva teórica, como un problema de **optimización sobre grafos**. En este modelo:



- Los nodos representan routers o dispositivos intermedios.
- Los enlaces entre ellos son los arcos del grafo, asociados con un **peso** que simboliza el costo o distancia de transmisión (por ejemplo, retardo, número de saltos o métrica definida por el protocolo).

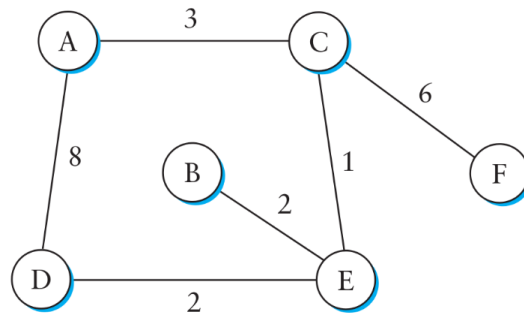
Resolver el ruteo implica encontrar el **camino de menor costo** entre un nodo origen y una red destino, aplicando algoritmos como *Dijkstra* (para protocolos de estado de enlace, como OSPF) o *Bellman-Ford* (para protocolos de vector de distancia, como RIP).

Sin embargo, a diferencia del modelo matemático puro, en **Internet** el objetivo del ruteo no es alcanzar un router específico, sino **alcanzar una red destino**. Cada router se encarga de determinar cuál es el siguiente salto óptimo que acerque el paquete a esa red, utilizando la información de su tabla de routing.

De esta manera, el ruteo proporciona el contexto y las decisiones estratégicas, mientras que el forwarding —tanto en routers como en switches— ejecuta de forma eficiente las acciones físicas que permiten el envío de datos dentro de la red.

2 Ejercicio 1

En la red de la figura los enlaces están etiquetados con los costos relativos.



Mostrar la tabla de forwarding para cada nodo. Cada tabla en cada nodo debe reflejar la ruta de menor costo para el envío de un paquete a un determinado destino.

Nodo A

Destino	Próximo Salto
A	-
B	C
C	C
D	C
E	C
F	C

Nodo B

Destino	Próximo Salto
A	E
B	-
C	E
D	E
E	E
F	E

Nodo C

Destino	Próximo Salto
A	A
B	E
C	-
D	E
E	E
F	F

Nodo D

Destino	Próximo Salto
A	E
B	E
C	E
D	-
E	E
F	E

Nodo E

Destino	Próximo Salto
A	C
B	B
C	C
D	D
E	-
F	C

Nodo F

Destino	Próximo Salto
A	C
B	C
C	C
D	C
E	C
F	-

¿De qué maneras se pueden llenar esas tablas? Mencione las diferencias más significativas.

En una red, el proceso de **enrutamiento (routing)** define el camino que deben seguir los paquetes desde su origen hasta el destino final. Este proceso puede implementarse de distintas maneras según el tamaño, la topología y las necesidades de administración de la red. Las dos formas principales de enrutamiento son el **enrutamiento estático** y el **enrutamiento dinámico**.

Enrutamiento Estático

El **enrutamiento estático** consiste en la configuración manual de las rutas en cada router de la red. Cada entrada de la tabla de enrutamiento debe ser establecida explícitamente por el administrador, indicando el destino, la máscara de red y el próximo salto o interfaz de salida.

Este método no requiere intercambio de información entre routers, lo que lo hace sencillo y predecible. Sin embargo, presenta limitaciones importantes en entornos grandes o cambiantes:

- Genera una **carga administrativa significativa**, ya que el mantenimiento y la actualización de las rutas deben hacerse manualmente en cada dispositivo.
- Los routers **no comparten su tabla de enrutamiento** con sus vecinos.
- **No tienen capacidad de reacción** ante fallos o cambios en la topología: si un enlace se cae, el tráfico seguirá intentando usar esa ruta, provocando pérdida de conectividad.

Por ello, el enrutamiento estático se utiliza principalmente en redes pequeñas, donde la topología es estable y las rutas son pocas y conocidas.

Enrutamiento Dinámico

El **enrutamiento dinámico** automatiza el proceso de descubrimiento y mantenimiento de rutas mediante el uso de **protocolos de enrutamiento** (como RIP, OSPF, EIGRP o BGP). Los routers intercambian información sobre la estructura de la red y ajustan sus tablas de enrutamiento en función de los cambios detectados.

- Los routers **comparten periódicamente su tabla de enrutamiento** con los routers vecinos, permitiendo que todos los dispositivos conozcan las mejores rutas disponibles.
- **Tienen capacidad de reacción** ante fallos o cambios en la red: si un enlace o nodo cae, los protocolos actualizan automáticamente las rutas, buscando caminos alternativos.
- **Reduce la carga administrativa**, ya que los ajustes se realizan de forma automática.

Sin embargo, el enrutamiento dinámico también presenta desafíos:

- Consume **recursos de procesamiento y ancho de banda**, ya que requiere intercambiar mensajes de control y mantener cálculos constantes.
- La convergencia ante fallos puede tomar tiempo, especialmente en redes grandes o complejas.
- Puede ser más difícil de depurar o controlar en comparación con el enrutamiento estático.

3 Protocolos de Ruteo Interno

3.1 Vector de Distancia - RIP

El **Routing Information Protocol (RIP)** es uno de los protocolos más representativos del *enrutamiento por vector de distancia*. Su objetivo es permitir que cada router conozca el camino más corto hacia las distintas redes de destino, a través de un proceso iterativo y distribuido basado en el intercambio de información con los routers vecinos.

3.1.1 Características generales

El enfoque de **Vector de Distancia** se basa en que cada router:

- Mantiene una tabla que indica, para cada red conocida, la distancia (en saltos) y el próximo salto o *next hop*.
- Envía periódicamente esta información a sus vecinos directos.
- Aprende sobre el resto de la red a partir de la información que recibe de sus vecinos.

En el caso de RIP, **cada nodo envía información de toda la red únicamente a sus vecinos directos**. Esto lo diferencia de los protocolos de *estado de enlace*, donde se difunde información de forma más global.

3.1.2 Paquete RIP

Cada nodo construye un vector que contiene pares del tipo:

$\langle \text{Red}, \text{Costo} \rangle$

Este vector representa el costo (habitualmente en cantidad de saltos) hacia cada red conocida. El paquete RIP transporta esta información, que los routers vecinos utilizan para actualizar sus propias tablas de enrutamiento. De esta forma, cada nodo mantiene una visión local del estado de la red, pero logra converger hacia rutas óptimas a través de la cooperación con sus vecinos.

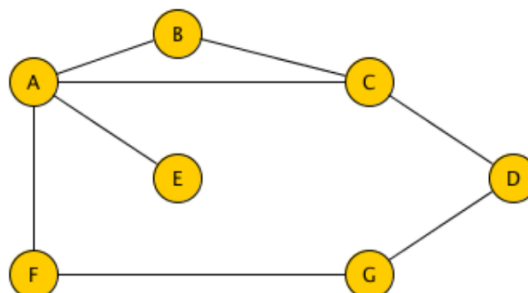
3.1.3 Funcionamiento del algoritmo

El algoritmo de Vector de Distancia sigue un proceso iterativo, en el cual cada nodo actualiza su conocimiento de la red a partir de los vectores recibidos. El procedimiento puede resumirse en los siguientes pasos:

1. **Inicialización:** Cada nodo construye un vector conteniendo la distancia a todos los demás nodos. Al inicio, se asume que el nodo sólo conoce a sus vecinos inmediatos, cuya distancia es 1. Para todos los demás destinos desconocidos, la distancia inicial es ∞ .
2. **Distribución:** Cada nodo distribuye su vector de distancias a todos sus vecinos inmediatos. Este intercambio ocurre de manera periódica y continua, para mantener actualizadas las tablas.
3. **Actualización:** Cuando un nodo recibe el vector de un vecino, aplica la siguiente regla:
 - Suma 1 a la distancia que el vecino reporta para cada red (el costo de alcanzarlo).
 - Si la distancia resultante hacia una red es **menor** que la registrada, actualiza su tabla de enrutamiento (aprende una mejor ruta).
 - Si la nueva distancia no mejora la existente, la información se descarta.
4. **Convergencia:** Este proceso continúa hasta que ningún nodo cambia su tabla de enrutamiento; en ese punto se dice que la red *converge*.

3.1.4 Ejercicio 2

Mostrar los mensajes RIP generados por el nodo A y la matriz global de Vector de Distancia en los siguientes escenarios:



Los nodos recién bootean y solo conocen las distancias de sus vecinos inmediatos.

Se asume que los nodos conocen únicamente a sus vecinos inmediatos.

Matriz de distancias inicial

Nodo	A	B	C	D	E	F	G
A	0	1	1	∞	1	1	∞
B	1	0	1	∞	∞	∞	∞
C	1	1	0	1	∞	∞	∞
D	∞	∞	1	0	∞	∞	1
E	1	∞	∞	∞	0	∞	∞
F	1	∞	∞	∞	∞	0	1
G	∞	∞	∞	1	∞	1	0

Vector de distancia de A (mensaje RIP inicial)

Red	Costo
A	0
B	1
C	1
E	1
F	1

Los nodos ya propagaron la información del inciso anterior.

Primera Iteración

Matriz de distancias después de la primera iteración

Nodo	A	B	C	D	E	F	G
A	0	1	1	2	1	1	2
B	1	0	1	2	2	2	∞
C	1	1	0	1	2	2	2
D	2	2	1	0	∞	2	1
E	1	2	2	∞	0	2	∞
F	1	2	2	2	2	0	1
G	2	∞	2	1	∞	1	0

Vector de distancia de A (mensaje RIP)

Red	Costo
A	0
B	1
C	1
D	2
E	1
F	1
G	2

3.1.5 La red ya convergió.

Segunda Iteración. En esta etapa, la red ya ha **convergió** y las tablas no cambian más con intercambios adicionales de mensajes RIP.

Nodo	A	B	C	D	E	F	G
A	0	1	1	2	1	1	2
B	1	0	1	2	2	2	3
C	1	1	0	1	2	2	2
D	2	2	1	0	3	2	1
E	1	2	2	3	0	2	3
F	1	2	2	2	2	0	1
G	2	3	2	1	3	1	0

Vector de Distancia de A (Mensaje RIP)

Red	Costo
A	0
B	1
C	1
D	2
E	1
F	1
G	2

3.1.6 Propiedades

1. **Rápida convergencia en redes estables:** En ausencia de cambios en la topología de la red, sólo se requieren unos pocos intercambios de mensajes entre los nodos para que cada router complete su tabla de enrutamiento y tenga conocimiento de todas las rutas disponibles.
2. **Métrica simple basada en saltos:** La distancia o costo se mide mediante la cantidad de saltos necesarios para alcanzar un destino. Cada enlace entre nodos se considera un salto unitario. Esta simplicidad facilita la implementación, aunque limita la granularidad y precisión en redes grandes o con enlaces de diferentes capacidades.
3. **Algoritmo distribuido:** Cada nodo trabaja de manera independiente con la información local que posee, pero colabora indirectamente con sus vecinos mediante el intercambio de vectores de distancia. Esta cooperación permite que la red, en conjunto, resuelva el problema global de encontrar rutas óptimas hacia todos los destinos.

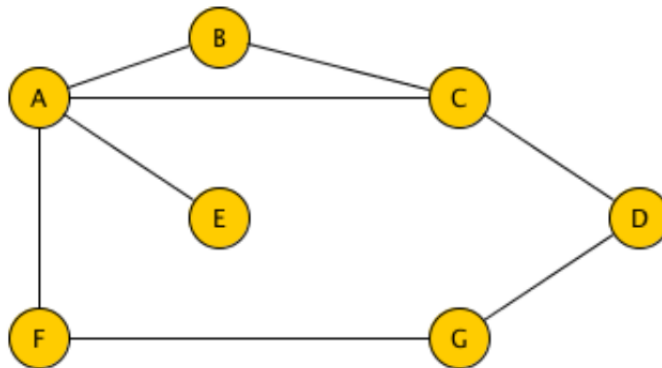
3.1.7 Actualizaciones de la tabla de enrutamiento

RIP implementa dos tipos de actualizaciones para mantener la tabla de enrutamiento:

- **Periódicas:** Cada nodo envía automáticamente un mensaje de actualización a intervalos regulares, incluso si no se ha producido ningún cambio en la topología. Esto garantiza que las tablas se mantengan sincronizadas y que cualquier información perdida por un nodo caído o un paquete perdido pueda recuperarse.
- **Disparadas (Triggered Updates):** Cuando ocurre un cambio en la red, como la caída de un enlace o la llegada de un mensaje que modifica la tabla, se envía inmediatamente un nuevo mensaje de actualización a los vecinos. Esto acelera la propagación de cambios importantes y mejora la capacidad de adaptación del protocolo.

3.1.8 Problema de conteo al infinito

Uno de los problemas más conocidos de los algoritmos de Vector de Distancia es el **conteo al infinito**. Se produce cuando un enlace falla, y los nodos vecinos calculan rutas alternativas basadas en información desactualizada:



- Por ejemplo, si el enlace entre los nodos A y E falla, A comunica a B y C que la distancia a E es infinita. Sin embargo, B y C todavía tienen información de que el otro nodo puede alcanzar E en 2 saltos.
- Cada nodo recalcula su distancia basándose en la información del vecino, incrementando el conteo en cada iteración. Esto provoca que los nodos sigan incrementando indefinidamente la distancia hacia el destino fallido, generando bucles de enrutamiento temporales y retrasando la convergencia.

Soluciones parciales al conteo al infinito

Se han desarrollado varias técnicas para mitigar este problema, aunque ninguna lo elimina completamente en redes grandes:

1. **NO-INF:** Se reemplaza el concepto de infinito por un número finito alto (por ejemplo, 16 en RIP). Esto limita el número máximo de saltos y evita que el conteo crezca indefinidamente, aunque no previene completamente los bucles temporales.
2. **Split Horizon:** Un nodo no envía actualizaciones de rutas hacia el vecino que le enseñó esa ruta. Esto impide que la información errónea se retroalimente inmediatamente entre dos nodos, reduciendo la probabilidad de conteo al infinito entre pares.
3. **Split Horizon con Poison Reverse:** En lugar de no enviar la ruta al vecino que la enseñó, el nodo envía una información negativa (por ejemplo, indicando infinito) sobre esa ruta. Esto asegura que el vecino sepa que la ruta ya no es válida.

3.2 Ejercicio 3 : Topología de Red

En una red un router tiene solo dos interfaces configuradas con las direcciones IP y máscaras 10.0.2.1/24 y 10.0.3.1/30. A su vez, tiene directamente conectado un solo router vecino, del cuál recibe periódicamente un paquete RIP con la siguiente información de ruteo:

Red	Máscara	Costo
10.0.2.0	255.255.255.0	1
10.0.3.0	255.255.255.252	0
10.0.4.0	255.255.255.0	0
10.0.5.0	255.255.255.0	1

Muestre una posible topología de red que se pueda deducir de la configuración del router y la información de ruteo que recibe.

Del enunciado podemos deducir



Los routers R1 y R2 tienen una red en común



Pero debemos ubicar una cuarta red, la 10.0.5.0/24



Muestre un posible paquete RIP que envía el router que recibe la información de ruteo.

Red	Costo / Métrica
10.0.2.0/24	0
10.0.3.0/30	0
10.0.4.0/24	1
10.0.5.0/24	2

Mostrar la tabla de forwarding del router que recibe la información de ruteo.

Red	Próximo Salto / Interfaz
10.0.2.0/24	IF0/0
10.0.3.0/30	IF0/1
10.0.4.0/24	10.0.3.2
10.0.5.0/24	10.0.3.2

3.3 Estado de enlace - OSPF

El protocolo **OSPF (Open Shortest Path First)** es un ejemplo clásico de algoritmo de *Estado de Enlace (Link-State)*, ampliamente usado en redes IP para enrutamiento interno. A diferencia de los protocolos de vector de distancia como RIP, OSPF permite a cada nodo tener una **visión completa de la topología de la red**, lo que le permite calcular rutas óptimas mediante algoritmos de grafos como Dijkstra.

3.3.1 Características principales

1. **Distribución de información:** Cada nodo envía información solo sobre sus *vecinos directos*, pero esta información se propaga a **toda la red** mediante el protocolo. De esta forma, cada router puede construir una **base de datos completa de topología** sin depender de la información de rutas completas de sus vecinos.
2. **Paquete OSPF:** Para comunicar el estado de sus enlaces, cada nodo genera un paquete que contiene un vector con los siguientes campos:

⟨ID nodo, SeqNum, TTL, Lista de vecinos⟩

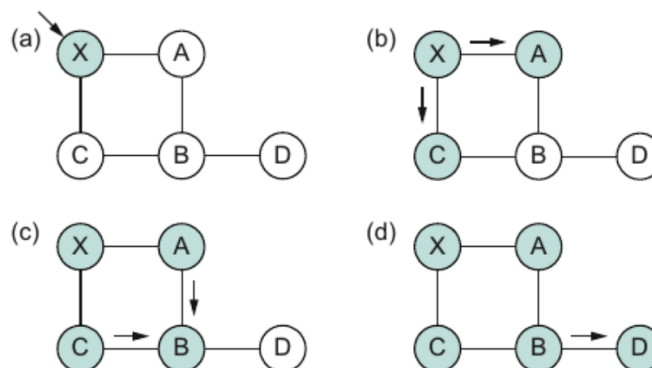
Adicionalmente, cada enlace reporta:

- La **red** a la que está conectado
- El **costo** asociado al enlace (métrica basada en ancho de banda u otros factores)

Este paquete se envía a todos los vecinos, quienes a su vez lo retransmiten, asegurando que la información llegue a todos los routers de la red.

3. **ID del nodo:** Identifica de manera única al router que envía el paquete, permitiendo distinguir entre diferentes fuentes de información en la red.
4. **Número de Secuencia (SeqNum):** Permite que los routers detecten versiones más recientes de la información de estado de enlace. Evita que se procesen datos antiguos o duplicados, lo que es fundamental para mantener la **consistencia de la base de datos** de topología.
5. **TTL (Time To Live):** Garantiza que los paquetes OSPF no se propaguen indefinidamente en la red. Cada vez que el paquete atraviesa un router, el TTL se decrementa; cuando llega a 0, el paquete se descarta. Esto evita bucles de retransmisión y reduce el tráfico innecesario.

Inundación confiable - Definición gráfica



Fuente: Larry L. Peterson and Bruce S. Davie. 2011.
Computer Networks, Fifth edition: A Systems Approach.

3.3.2 Funcionamiento

1. **Descubrimiento de vecinos:** Cada router identifica automáticamente quiénes son sus *vecinos directos*, mediante mensajes de descubrimiento. Además, aprende las *direcciones de red* asociadas a cada vecino, información esencial para construir la topología de la red.
2. **Determinación de la métrica de los enlaces:** Para cada vecino, el router establece un *costo o métrica* que representa la “distancia” o “dificultad” de alcanzar ese enlace. Esta métrica puede basarse en factores como ancho de banda, retardo, confiabilidad o número de saltos, dependiendo de la configuración de la red.
3. **Construcción del paquete de estado de enlace (LSA):** Con la información recopilada, el router genera un *paquete LSA (Link State Advertisement)* que resume todo lo que ha aprendido sobre sus vecinos y enlaces. Este paquete incluye campos como: identificación del nodo, lista de vecinos, métricas de enlace y número de secuencia para garantizar que la información más reciente se propague correctamente.
4. **Propagación de información:** El router envía el paquete a **todos los demás routers** de la red, y a su vez recibe paquetes de los otros routers. Gracias a este intercambio, cada router construye una **base de datos completa de la topología**, que refleja todas las conexiones entre los nodos de la red.
5. **Cálculo de rutas óptimas:** Una vez que la base de datos de topología está actualizada, el router aplica un *algoritmo de ruta más corta* (generalmente Dijkstra) para determinar la **mejor ruta hacia cada destino** en la red. La tabla resultante se utiliza para el **reenvío eficiente de paquetes**, garantizando que el tráfico siga siempre el camino más corto o más eficiente según la métrica establecida.

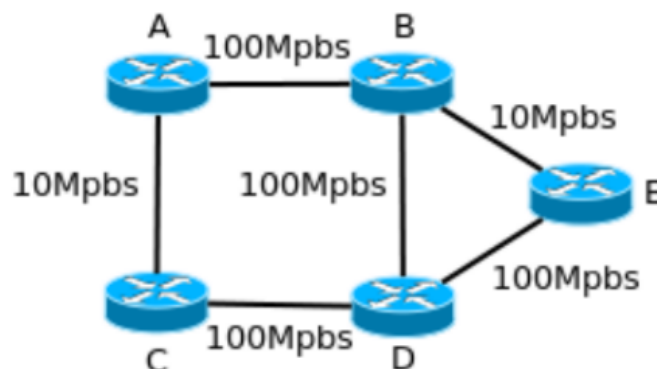
Métricas en Protocolos de Enrutamiento

En el contexto de ruteo, la **métrica** es un valor numérico que indica qué tan “cara” o “corta” es una ruta hacia un destino. Dependiendo del protocolo, la forma de calcularla y su significado pueden variar.

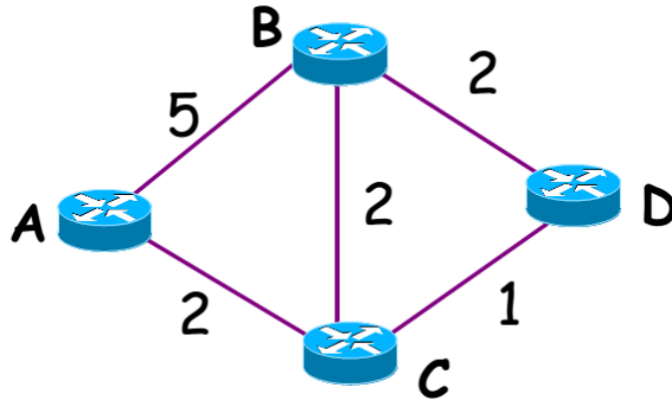
- Métricas de RIP
 - En **RIP (Routing Information Protocol)**, la métrica se mide únicamente como el número de saltos (*hop count*) entre el router origen y el destino.
 - Ejemplo: **Métrica = 1**, significa que la red de destino está a un salto directo del router.
 - RIP no considera ancho de banda, retardo ni otros factores; sólo la cantidad de routers intermedios.
- Métricas de OSPF
 - **OSPF (Open Shortest Path First)** utiliza un cálculo más flexible, generalmente basado en el **ancho de banda del enlace**:

$$\text{Métrica OSPF} = \frac{100\,000\,000 \text{ (ó } 10^{10} \text{)}}{\text{ancho de banda del enlace en bps}}$$

- En el ejemplo dado, la métrica es **1010**, lo que indica que el enlace tiene un ancho de banda determinado (aproximadamente 100 Mbps según la fórmula de OSPF).
- En OSPF, una métrica más baja significa una mejor ruta (más rápida o con mayor capacidad).



3.3.3 Ejercicio 4 : OSPF



Dada la red de la figura, suponer que el protocolo de ruteo es utilizado es OSPF. Se pide:

Mostrar todos los mensajes (el contenido de los campos relevantes) que recibe A hasta que la red converge.

De B recibe

- ID: B
- TTL: X
- Número de Secuencia: Z

Nodo	Costo
A	5
C	2
D	2

De C recibe

- ID: C
- TTL: X
- Número de Secuencia: Y

Nodo	Costo
A	2
B	2
D	1

De D recibe

- ID: D
- TTL: X-1
- Número de Secuencia: W

Nodo	Costo
B	2
C	1

De D recibe un TTL menos ya que entre los nodos A y D hay otro router.

Aclaraciones importantes

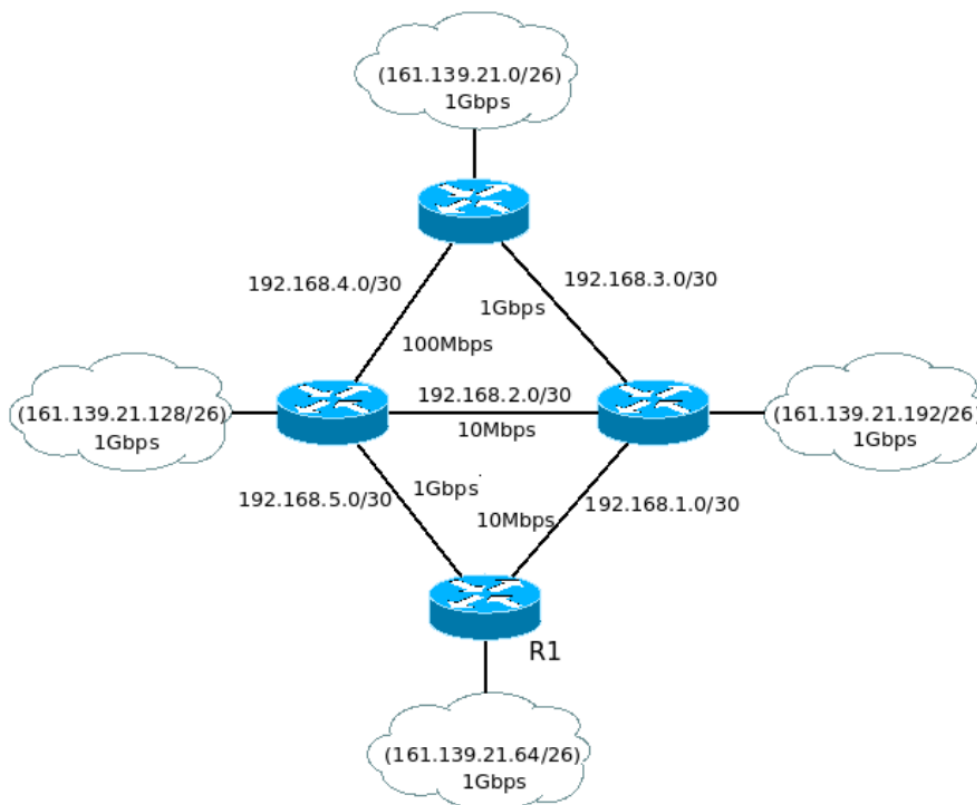
- El TTL de D se reduce en 1 porque entre los nodos A y D hay otro router intermedio.
- El algoritmo establece que, en todo momento, se envía la **información de la distancia directa a sus vecinos**. Por lo tanto, salvo el número de secuencia y TTL, la demás información en los mensajes OSPF no varía entre iteraciones.
- Lo que sí cambia con cada iteración es el **estado de la tabla de ruteo de cada nodo**.

Explicar cómo A construye su tabla de ruteo a partir de los mensajes recibidos.

(confirmado)	tentativo	
(A,0,-)		inicializo conmigo mismo
(A,0,-)	(B,5,B) (C,2,C)	traigo a tentativo los vecinos de A
(A,0,-) (C,2,C)	(B,5,B)	muevo a confirmado el de menor costo (C)
(A,0,-) (C,2,C)	(B,4,C) (D,3,C)	traigo los vecinos de C con pesos actualizados y reemplazo
(A,0,-) (C,2,C) (D,3,C)	(B,4,C)	traigo los vecinos de C con nuevo el de menor costo (D)
A,0,- (C,2,C) (D,3,C) (B,4,C)		ninguna vecino de C agrega o mejora lo que hay en tentativo, así que muevo B a confirmado y termino

3.3.4 Ejercicio 5

En la red de la figura se interconectan 4 subredes mediante 4 routers, que corren el protocolos OSPF.



Muestre el mensajes OSPF que envía el router R1 al resto.

- ID: R1
- TTL: X
- Número de Secuencia: W

Red	Costo
161.139.21.64/26	10
192.168.5.0/30	10
192.168.1.0/30	1000

Mostrar la tabla de forwarding para para el router R1. Asignar direcciones IPs a todas las interfaces que haga falta.

Red	Próximo Salto / Interfaz
161.139.21.64/26	IF0/0
192.168.5.0/30	IF0/1
192.168.1.0/30	IF0/2
161.139.21.128/26	192.168.5.1
161.139.21.0/26	192.168.5.1
161.139.21.192/26	192.168.5.1
192.168.2.0/30	192.168.5.1
192.168.3.0/30	192.168.5.1
192.168.4.0/30	192.168.5.1